

PrismatAI Training Plan

Version 3 — Release 2 (V3-R2)

Date	2026-03-09
Status	FINAL — reviewed (9 external reviews + 2 meta-reviews), ready to implement
Approach	Track A — Supervised learning on human replays first; iterative self-play in a future phase
Author	Surfinite + Claude Code

Goal

Train a neural network evaluation function for Prismata that, when used inside the existing Alpha-Beta Hierarchical Portfolio Search (HPS) engine, produces an AI stronger than MasterBot (the Steam client's native AI).

Key numbers

- **102,697** usable training games (both players ≥ 1500 , rated, balance-validated)
- **116** competitive units (11 base + 105 dominion)
- **~3.1M** raw training records (start-of-turn states only)
- **1,290** features per state (Option 2 — type-count + instance flags + supply + card-set indicator)
- **740** unique players; mean combined rating $\sim 3,909$

Table of Contents

Key Verification Items (V1–V9)

Overview

Phase 0: Prerequisites and Baseline Measurements

- 0a. Complete Balance Validation [DONE]
- 0b. Benchmark Tournament [TODO]
- 0c. JS Engine Validation
- 0d. Dataset Composition Audit
- 0e. C++ Inference Speed Benchmark
- 0f. Optional: cardLibrary Cleanup

Phase 1: Feature Schema Design

- 1a. The Unit Representation Problem
- 1b. Label Design
- 1c. Schema Versioning

Phase 2: Data Extraction Pipeline

- 2a. Extraction
- 2b. Train / Validation / Test Split
- 2c. Deduplication
- 2d. Post-Extraction Validation

Phase 3: Architecture Search and Ablation

Phase 4: Full Training Run

Phase 5: Evaluation

Phase 6: Self-Play Iteration (Future)

Opening Book

Known Failure Modes

Resolved Design Questions

Optional Enhancements — Decisions

References

Key Verification Items

These are facts the plan depends on. If any are wrong, assumptions about data scale, feasibility, or success criteria may need revision.

V1 — Usable Human Replay Count (validated, final)

Full balance validation (cost + rarity against current `cardLibrary.json`) completed across **all 203,602 codes** on March 7, 2026. Results: **154,061** balance-passed, 48,707 failed, 834 fetch errors (S3 404s). After applying the full quality filter (both players ≥ 1500 rating at game time, rating change $\neq 0$ to exclude friendly/unrated games): **102,697 usable training games**. 740 unique players contribute. Post-patch pass rate: $\sim 97.8\%$. Pre-patch pass rate: $\sim 57.7\%$.

V2 — Competitive Unit Count

The authoritative unit list is `bin/asset/config/valid_units.json` and contains exactly **116 units**: 11 base set units (present in every game) and 105 dominion/random set units. The `cardLibrary.json` contains 161 entries but 45 of those are deprecated, event-mode, or removed units. Any feature schema must be built on the **116-unit list**, not the raw `cardLibrary` count.

V3 — Balance Validation Definition

A replay is considered balance-valid if its unit costs **and** unit rarities (supply limits) match the current `cardLibrary.json`. The validation script (`validate_db_codes.py`) checks both `buyCost` and `rarity` for every unit in every replay.

Decision: validate all replays at training time. The January 14, 2019 cutoff was previously used as a fast filter. This approach is fragile. Instead, every replay in the training set must pass the full cost+rarity validation before use, regardless of timestamp.

Rarity audit results (March 2026): 17 units with historical rarity changes identified. Most notable: **Bombarder** (normal $\rightarrow$ rare, supply 20 $\rightarrow$ 4), affecting ~ 544 training-eligible replays. 16 other units had mostly trinket $\rightarrow$ normal or trinket $\rightarrow$ legendary transitions clustered around November 2015.

V4 — JS Engine Accuracy

The JavaScript engine is a transpilation of the original AS3 engine and is considered authoritative. Previously validated (100% pass rate on 500 replays), but **revalidation required** — the JS engine code was modified during recent integration work. During Phase 2a extraction, every replay will be validated as it passes through the engine.

V5 — MasterBot as Primary Benchmark

Target: the trained model, used as the evaluation function in Alpha-Beta HPS search, **beats MasterBot** (the Steam client's native AI). Access is via **SteamAI** (`js_engine/steam_ai.js`), which spawns Steam's native `PrismataAI.exe` per turn. Fully integrated into the matchup runner: `node matchup_clean.js --player SteamAI --games N`.

V6 — Dataset Composition, Record Scale, and Supplementary Data

- **PvP status confirmed:** All 102,697 usable training games are PvP. MasterBot (rating ~1.0) excluded by the rating ≥ 1500 filter.
- **Record count estimate:** ~ 30 turns average game length $\times$ 102,697 games = **$\sim 3.1\text{M}$ raw training records** (one record per player-turn, start-of-turn states only). Mid-turn states excluded — they introduce distribution shift and label ambiguity.
- **Grey data** — human vs MasterBot replays: a potentially large pool of supplementary data, but lower-confidence. Assess as a contingency data source.

V7 — Rating Distribution (measured, confirmed)

Mean combined rating **$\sim 3,909$** (per-player avg $\sim 1,954$). Distribution: 2000+ = 38.9%, 1800–1999 = 47.2%, 1600–1799 = 13.6%, 1500–1599 = 0.2%. The dataset is dominated by high-level play.

V8 — Unit Properties Available for Set-Based Encoding

All 105 random-pool units have structured properties in `cardLibrary.js`: buy cost, toughness, build time, default blocking, lifespan, fragile flag, and other numeric stats. Unit *abilities* are defined as scripts rather than numeric values — encoding them requires simplification or omission.

V9 — C++ AI $\leftrightarrow$ JS Engine Click Protocol Correctness

Hard gate for Phase 5 and Phase 6. Failures manifest as `skippedBuys` or incorrect game state. All move types must work: BUY, USE_ABILITY (including shift-click), UNDO_USE_ABILITY, ASSIGN_BLOCKER, SNIPE/CHILL (two-step targeting), END_PHASE.

Verification: Run 512+ game LiveHardestAI vs MasterBot matchup. Confirm zero `skippedBuys`, zero click failures for all move types, plausible game outcomes.

Failure triage: BUY failures $\rightarrow$ check `Benchmarks.cpp` click emission; USE_ABILITY $\rightarrow$ check C++ multi-card expansion vs JS single-click; SNIPE/CHILL $\rightarrow$ check two-step targeting; ASSIGN_BLOCKER $\rightarrow$ check inert-card blocking rules.

Overview

The goal is to train a neural network evaluation function for Prismata that, when used inside the existing Alpha-Beta Hierarchical Portfolio Search (HPS) engine, produces an AI stronger than MasterBot. Churchill (2019) demonstrated this approach works — training on 500K AI self-play games produced an evaluator that beat both the resource heuristic and the playout evaluator. We build on that result with three key differences:

- **Accurate engine:** The JS engine correctly implements all game rules. Prior C++ approximations had known errors.
- **Human replay data:** Human expert games carry stronger strategic signal than AI self-play.
- **Architecture exploration:** Churchill's 2-layer, 512-neuron MLP was chosen without exhaustive search. We will run targeted ablations.

Known limitation — distribution shift: A model trained on expert human positions will be evaluated on positions generated by AI search — which may include states humans rarely reach. This is the primary motivation for Phase 6 (self-play iteration). Phase 5 evaluation should include qualitative review of cases where the model's evaluation appears confident but strategically wrong.

Self-play iteration (AlphaZero-style) is planned as a future phase. This plan covers supervised training only.

Phase 0: Prerequisites and Baseline Measurements

Complete all setup before any training work begins. Nothing in Phase 1+ should start until all items here are done and recorded.

0a. Complete Balance Validation

STATUS: DONE — Completed March 7, 2026

- Full re-validation of all 203,602 codes — single clean run with current cost+rarity validator.
- Results: 154,061 balance-passed, 48,707 failed, 834 S3 fetch errors.
- Results imported into `replays.db`. Final usable training set: **102,697 games**.
- Pending: export canonical code list `balance_validated_1500plus.json` from DB.

0b. Benchmark Tournament

STATUS: TODO

- Run 512+ game **LiveHardestAI vs MasterBot** matchup via SteamAI. Record win rate.
- This run also serves as V9 click-protocol verification — confirm zero skippedBuys and zero click failures.
- **SteamAI determinism check:** Run the same game state through MasterBot multiple times (≥ 10 identical positions). If deterministic, paired matches produce correlated results requiring adjusted statistical analysis.
- Human expert visual review of a sample of games for strategic coherence.

0c. JS Engine Validation on Representative Sample

- Run replay validation on 1,000 fresh random balance-validated codes as a pre-extraction sanity check.
- **Decision gate:** If pass rate below 95%, investigate before proceeding.
- During Phase 2a, every replay validated as it passes through the engine — this sample check is a fast pre-flight.

0d. Dataset Composition Audit

- PvP vs PvAI ratio; rating distribution; game length distribution; card set distribution
- **Draw rate:** If draws exceed 2%, consider label = 0.5 handling.
- **First-player win rate:** If P1/P2 deviation $> 2pp$, record for Elo prior formula.
- **Resignation/timeout handling:** If $> 5\%$ of games end by resignation, trim last 3–5 pre-resignation positions by default. Run one ablation without trimming to quantify the effect.
- **Per-unit frequency:** Flag units in < 500 games as low-data units for separate Phase 5 reporting.
- **Turn-bucketed label entropy:** Report by turn bucket (turns 1–8, 9–18, 19+) to quantify noise.

0e. C++ Inference Speed Benchmark

Before Phase 3 architecture decisions are finalised, benchmark the C++ inference engine at different model sizes. The trade-off — model quality vs. search depth — is the central tension.

- Benchmark sizes: 128h/2L, 256h/2L, 256h/3L, 512h/2L, 1024h/2L, plus residual variants 256h/4R, 128h/4R.
- **Benchmark in actual search loop** at 3s and 7s think times. Record: raw evals/sec, nodes searched, effective search depth. Also measure with a **constant evaluator** to establish the search budget ceiling.
- **Derive the minimum eval speed** from measured search budget — the previous 500 evals/sec heuristic was a guess.

0f. Optional: cardLibrary Cleanup

- Remove 45 non-competitive unit entries from cardLibrary.json; update referencing code.
- Independent of training pipeline — can be parallelised with other Phase 0 work.

Phase 1: Feature Schema Design

Define a canonical, versioned feature vector before any data extraction begins. This schema must be agreed upon and frozen before Phase 2 starts. Changing the schema after extraction requires full re-extraction.

1a. The Unit Representation Problem

A game uses 11 base set units plus 8 randomly selected from the 105-unit dominion pool — at most 19 unit types per game. The feature vector must accommodate all 116 possible unit types.

Option 1 — Type-Count (Churchill 2019 Baseline)

- Feature vector: ~**245 features** (116×2 counts + 12 resources + 1 player indicator)
- Simple, fast inference, established precedent. Loses all instance-level information.

Option 2 — Type-Count with Instance Flags, Supply, and Card-Set Indicator (Selected)

SELECTED — matches the feature layout already implemented in `NeuralNet.cpp`.

For each of the 116 unit types, encode 11 values:

- P0 ready / constructing / exhausted / blocking count (4 values)
- P1 ready / constructing / exhausted / blocking count (4 values)
- P0 supply remaining, P1 supply remaining
- In card set (1.0 if available for purchase, 0.0 otherwise)

Plus 14 global features: 6 resource types × 2 players + turn number + active player indicator.

Feature vector size: $116 \times 11 + 14 = \mathbf{1,290 \text{ features}}$

Options 3 (per-instance encoding) and 4 (set-based/property encoding) are deferred to future iterations.

1b. Label Design

Training target: probability that Player 0 (P0) wins the game from this state. P0 is always the player whose features appear first (game index 0). The active player indicator is an input feature, NOT a selector for whose win probability to predict.

Canonical loss/output/inference path (Path A — logit/probability-native):

- **Training:** Model outputs raw scalar logit z . Loss = BCEWithLogitsLoss(z , y) where $y \in [0, 1]$.
- **Python validation:** $p = \text{sigmoid}(z)$ for probability interpretation.
- **C++ export:** Apply $v = 2 \cdot \text{sigmoid}(z) - 1$ for the $[-1, 1]$ signed value Eval.cpp expects, then scale.
- Replace existing `tanhf()` in C++ with `2.0f * sigmoidf(rawValue) - 1.0f`.

The Noisy Label Problem (Critical)

Binary game outcomes are very noisy for early-turn positions. A turn-1 position from a game P1 won gets label 1.0, but the true win probability is close to 0.5. More acute for high-rated human games (longer, closer) than Churchill's AI self-play.

Label strategy: 4-way ablation (Phase 3)

Strategy	Description	Notes
A — Hard binary	Raw 0/1 labels	Simplest baseline; model must average all noise
B — Temporal weighting	0/1 labels with sample weight by game progress ($0.3 + 0.7t$)	Downweights noisy early positions without modifying labels
C — Elo-interpolated	Blend outcome with Elo prior: $(1-t) \cdot \text{elo_prior} + t \cdot \text{outcome}$	Higher ceiling; uses fixed REFERENCE_LENGTH=40, not total_turns
D — Neutral 0.5-prior	$(1-t) \cdot 0.5 + t \cdot \text{outcome}$	Simpler than C; avoids injecting player-skill info into target

If C and D produce similar results, prefer D (simpler, no external data dependency).

1c. Schema Versioning

- Schema stored as `training/schema_v1.json` — any change = new version.
- Must include: player convention, value target, unit list, feature names/indices, value ranges, normalization constants, label strategy.
- **Include a schema hash/checksum** in output filenames and binary shard headers to prevent accidental mixing.

Phase 2: Data Extraction Pipeline

2a. Extraction

- Input: `balance_validated_1500plus.json` from Phase 0a
- Process: fetch each replay from S3, step through via JS engine, record one training record per player-turn at start-of-turn states only.
- Output: binary shard format (or HDF5). JSONL at 3.1M records (~15–29 GB) is impractically slow.
- Record metadata: `replay_code`, `turn_number`, `rating_p0`, `rating_p1`, `game_date`, `card_set`, `total_turns`, `ply_index`.
- Store metadata in a **separate block from the feature vector**. Add extraction test verifying no metadata in feature indices.
- Engine validates every replay during extraction; halt if failure rate >1%.
- Parallelization: 4–8 workers to reduce extraction from ~14h to ~2–4h.

Symmetry augmentation — NOT applied by default. Prismata has asymmetric starting positions (P0: 6 Drones, P1: 7 Drones). Naive player-swap creates unreachable early-game states. Turn-gated mirroring tested as a Phase 3 ablation.

- **Base-only game exclusion:** Excluded from primary training set (strategically distinct, no competitive relevance).

2b. Train / Validation / Test Split

- **Primary split: temporal.** Most recent 10% = test, next 10% = validation, remaining 80% = training.
- **Secondary diagnostic:** random 10% holdout for comparison — if temporal loss is much worse, model is memorizing historical patterns.
- If turn-gated mirroring adopted, apply AFTER the split.

2c. Deduplication

- Deduplicate by code before extraction. Verify no duplicate records post-extraction.

2d. Post-Extraction Validation

- Spot-check 100 random records against expected outcomes.
- Verify label distribution (~50/50), feature value ranges, total record count.
- **Three-way feature parity test** (JS → Python → C++): 100 states, all three must produce identical feature vectors.
- **Training data loader verification:** Load 100 records through actual PyTorch pipeline, decode back to human-readable, verify against extraction output.

- **Per-feature statistics:** Compute mean/std/min/max for every feature dimension. Flag zero-variance or implausible values.
- **Seat asymmetry audit:** Report P0-active vs P1-active counts, first-player win rate per split, verify starting Drone asymmetry in early-game stats.

Phase 3: Architecture Search and Ablation

Determine a good architecture before a full training run. Use a representative subset of training data. **Track ablation runs with W&B or MLflow.** Do not proceed to Phase 4 without reviewing results.

3a. Churchill Baseline (Lower Bound)

- Feature set: Option 1 (~245 features); Architecture: 2 hidden, 512 neurons, tanh; Adam LR=1e-5; MSE loss.
- This is the lower bound. If a more complex model cannot beat it, something is wrong.

3b. Feature Richness Ablation

- Train Churchill architecture on Option 2 features (~1,290). Compare validation loss.
- If Option 2 adds <1pp: assess complexity justification. If ≥2pp: Option 2 is the better starting point.

3c. Capacity Search and Architecture Variants

- Vary hidden dimension: 128, 256, 512, 1024. Vary depth: 2, 3, 4 layers.
- **Residual MLP blocks:** Include at least one run with skip connections (4-block residual). C++ already supports residual blocks with LayerNorm.
- **Ablation efficiency:** Use fixed epoch count (e.g., 30 epochs + early stopping) for architecture comparison. Apply Phase 0e inference speed constraint as a separate hard filter.

3c-2. Label Strategy Ablation

- Compare Strategies A, B, C, D on best architecture from 3c with same training budget.
- If C and D produce similar results, prefer D (simpler, no external data dependency).

3c-3. Sampling, Weighting, and Augmentation Ablations

- Uniform sampling vs. inverse-game-length weighting vs. per-game uniform sampling vs. capped per-game contribution
- **Rare-unit-aware sampling:** Oversample games with low-frequency units by 2–3×.
- **Mirror augmentation ablation:** No mirroring (baseline) vs. turn-gated mirroring vs. full mirroring.
- **Weight monitoring:** Log effective weight histogram; verify ≥80% of samples have weight >0.3.

3d. Policy Head Experiment

- Add a policy head predicting **buy counts per unit type** (not full-turn imitation). C++ PUCT infrastructure already supports this.
- Include if accuracy ≥20%; drop if <15%.

3e. Search-Position Sanity Check

- Before final architecture decision, verify top candidates behave sanely on AI-search-generated positions.
- Check value consistency, mirror symmetry, tactical sanity. Fast diagnostic (~10 min per candidate).

3f. Architecture Decision Gate

- Review: winning feature set, optimal depth/width, data vs capacity limitation, policy head value, label strategy, sampling strategy.
- **Decision rule:** If richer model loses $>0.1\text{ms/eval}$ but improves loss <0.005 , prefer simpler.
- **Mini-arena:** 64–128 paired games vs fixed baseline for top 2–3 candidates. Sanity check that offline metrics correlate with playing strength.

3g. Feature Importance Analysis

- Run SHAP or permutation importance on winning model. Report top/bottom 50 features.
- Diagnostic only — does not block Phase 4.

Phase 4: Full Training Run

4a. Training

- Full training set (80% temporal split). Validate every epoch; save best checkpoint by validation loss.
- **Gradient clipping**: max norm 1.0 as default — prevents exploding-gradient collapses with BCE + noisy labels.
- **SWA**: Maintain Stochastic Weight Averaging model alongside standard checkpoint (last ~20% of epochs).
- Early stopping with patience ~10 epochs.
- **Rating-based sample weighting**: $\text{weight} = ((p0_rating + p1_rating) / 4000)^2$. 2000+2000 $\rightarrow$ 1.00; 1800+1800 $\rightarrow$ 0.81; 1500+1500 $\rightarrow$ 0.56. Compare vs. unweighted in Phase 3.

4b. Regularisation

- Dropout: 0.1–0.3 on hidden layers; Weight decay: L2 (1e-4 to 1e-5).
- **Label smoothing**: Apply only if Strategy A is selected. Redundant with B/C/D.
- **Mixup**: Optional, low priority. Skepticism noted — convex combinations of game states are not legal. Test only after dropout/decay/SWA established.

4c. Learning Rate Schedule

- Cosine decay or step decay; LR warmup (500–2000 steps) to prevent early instability.

4d. Infrastructure

- Local: Intel Arc B580 (12GB VRAM, XPU backend). Cloud fallback: AWS g4dn.xlarge or GCP g2-standard-4.

4e. Export and C++ Integration

- Export to binary format compatible with NeuralNet.cpp (26 tensors required).
- **Cross-language verification**: ~10 canonical test states (opening, mid-game, late-game, empty, mirror pairs). Confirm outputs match to 4+ decimal places. Cast to float32 before export.
- Export normalization constants in `schema_v1.json` — verify they match C++.
- Test both pure NeuralNet mode ($\times 100$) and NeuralNetPlusPlyout blend mode.

4f. Tactical Regression Suite

- 20–50 known positions with expected qualitative behavior: forced lethal, safe defense, dominant buys, mirror-symmetric, free-resource probes, degenerate boards.
- Fast (~2 min) sanity check that catches integration bugs.

4g. Reproducibility Checklist

- Log: schema hash, commit hash, split manifest, random seed, opening-book setting, search params, normalization constants, export checksum, label/sampling strategy.

Phase 5: Evaluation

5a. Tournament Setup

- Primary opponent: **MasterBot** (Steam) via SteamAI.
- Games: **minimum 1,024** (3pp win rate difference detectable at 95% confidence). Paired matches (swap starting players per card set).
- Think time: **3 seconds per turn** (primary). Also report at 1s and 7s.
- Wilson confidence interval; paired statistical tests (McNemar's or sign test).
- **Turn-bucketed evaluation** and **card-set stratified evaluation**.
- Every primary result must note whether opening book was active.

5b. Success Criteria

Result	Criterion	Action
Pass	Win rate vs MasterBot >50% (lower CI bound >0.50)	Proceed to Phase 6
Partial	Above 50% but not statistically significant	Run more games
Fail	Below 50%	Investigate pipeline, architecture, data quality
Near-miss	45–48% win rate	Partial success — approach works, needs more data or self-play

5b-2. Offline Search-Position Sanity Check

- Run HPS + new eval on ~1,000 self-generated positions. Check value consistency, mirror symmetry, tactical sanity, value monotonicity probes.

5b-3. Search Integration Ablation

- Compare: Pure NeuralNet vs NeuralNetPlusPayout (3+ blend weights) vs Pure Payout.

5c. Secondary Evaluations

- Self-play (expected ~50%), LiveHardestAI reference, qualitative play sessions, calibration plots, move agreement rate, pairwise ranking accuracy.

5d. Decision Gate

- **Success:** Proceed to Phase 6 (self-play). **Failure:** Diagnose and return to Phase 3/1/2. **Partial:** Begin self-play with current model.

Opening Book

Current State

The HPS engine uses an opening book with two tiers extracted from the live game's AI parameters: **LiveOpeningBook** (4 general entries) and **LiveOpeningBook2** (50 state-specific entries covering 9 random-pool units). Hand-coded by Lunarch Studios.

Opportunity: Data-Driven Opening Book

The 102,697 expert replays provide a much larger and more empirically grounded source. For each card set, observe what 2000+ rated players actually do on turns 1–3.

- Covers all 105 random-pool units (vs 9 in current book)
- Reflects actual high-level play rather than designer intuition
- Strong openings reduce importance of early-game evaluation where noisy-label problem is worst

Recommended action: Extract as a standalone Phase 0 task. Every Phase 5 tournament result must report whether the opening book was active. Run at least one evaluation with and one without.

Phase 6: Self-Play Iteration (Future Phase — Outline Only)

6a. Self-Play Data Generation

- Use trained model as eval function in JS engine matchup runner. Generate $\geq 100,000$ self-play games.

6b. Iterative Training

- Retrain on human + self-play data mix. Maintain $\geq 20\%$ human data (AlphaZero practice). Evaluate each iteration.

6c. Gumbel AlphaZero Consideration

- Improved exploration via sampling without replacement (Danihelka et al., ICLR 2022). Directly applicable to HPS.

Known Failure Modes

Failure Mode	Detection	Mitigation
Mode collapse — constant ~0.5 output	Val loss plateaus immediately; narrow prediction histogram	Check labels, gradient flow, normalization
Overfitting to early game	Turn-bucketed val loss diverges by phase	Re-weight toward late game; use Strategy B
Card set memorization	Card-set stratified eval shows large seen/unseen gap	Consider property encoding (Option 4); more diverse data
Calibration collapse	Poor calibration plots; Brier score diverges from log-loss	Add calibration term; temperature scaling post-hoc
Distribution shift failure	Low tournament win rate despite good validation	Fundamental SL limitation — proceed to Phase 6
Feature extraction bug	Zero-variance features; implausible per-feature stats	Per-feature statistics in 2d; 3-way parity test

Resolved Design Questions

- **Feature richness:** Option 2 (type-count + instance flags + supply + card-set indicator). Option 1 as Phase 3a baseline.
- **Rating floor:** Keep 1500 floor with rating-based sample weighting. Only 0.2% of games in 1500–1599 bracket.
- **Card set filtering:** Exclude base-only games — strategically distinct, no competitive relevance.
- **Value target:** Binary win outcome with soft-label modification (Phase 1b).
- **Architecture prior:** Residual MLP blocks for 4+ layers; plain MLP fine at 2–3 layers.
- **Benchmark calibration:** MasterBot (Steam) as primary benchmark, supplemented with held-out prediction accuracy and move agreement.

Optional Enhancements — Decisions

#	Enhancement	Status
1	Player-heldout diagnostic split	Declined
2	Card-set holdout test	Declined
3	Recency weighting ablation	Declined
4	Curriculum learning on game phase	Declined
5	Feature importance analysis (SHAP/permutation)	Adopted → Phase 3g
6	Draw rate measurement	Adopted → Phase 0d
7	Pairwise ranking metric	Adopted → Phase 5c
8	SteamAI determinism check	Adopted → Phase 0b
9	Side-to-move canonicalization ablation	Declined
10	Softer rating weighting alternatives	Declined
11	Report loss-at-epoch and loss-at-time	Adopted → Phase 3c

References

- Churchill, D. & Buro, M. (2015). *Hierarchical Portfolio Search: Prismata's Robust AI Architecture for Games with Large Search Spaces*. AIIDE 2015.
- Churchill, D. (2019). *Machine Learning State Evaluation in Prismata*. AIIDE Workshop 2019.
- Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel*. ICLR 2022.
- Hubert, T. et al. (2021). *Learning and Planning in Complex Action Spaces (Sampled MuZero)*. ICML 2021.

- Landers, M. et al. (2025). *SAINT: Attention-Based Policies for Discrete Combinatorial Action Spaces*.
- Perolat, J. et al. (2022). *Mastering the Game of Stratego with Model-Free Multiagent Reinforcement Learning*. *Science*, 378(6623).